Detecting cycles in a graph using DFS: 2 different approaches and what's the difference

Note that a graph is represented as an adjacency list.

I've heard of 2 approaches to find a cycle in a graph:

1. Keep an array of boolean values to keep track of whether you visited a node before. If you run out of new nodes to go to (without hitting a node you have already been), then just backtrack and try a different branch.
2. The one from Cormen's CLRS or Skiena: For depth-first search in undirected graphs, there are two types of edges, tree and back. The graph has a cycle if and only if there exists a back edge.

Can somebody explain what are the back edges of a graph and what's the diffirence between the above 2 methods.

Thanks.

Update: Here's the code to detect cycles in both cases. Graph is a simple class that represents all graph-nodes as unique numbers for simplicity, each node has its adjacent neighboring nodes (g.getAdjacentNodes(int)):

```
public class Graph {

  private int[][] nodes; // all nodes; e.g. int[][] nodes = {{1,2,3}, {3,2,1,5,6}...};

  public int[] getAdjacentNodes(int v) {
    return nodes[v];
  }

  // number of vertices in a graph
  public int vSize() {
    return nodes.length;
  }

}
```

Java code to detect cycles in an undirected graph:

```
    public class DFSCycle {

    private boolean marked[];
    private int s;
    private Graph g;
    private boolean hasCycle;

    // s - starting node
    public DFSCycle(Graph g, int s) {
        this.g = g;
        this.s = s;
        marked = new boolean[g.vSize()];
        findCycle(g,s,s);
    }

    public boolean hasCycle() {
        return hasCycle;
    }

    public void findCycle(Graph g, int v, int u) {

        marked[v] = true;

        for (int w : g.getAdjacentNodes(v)) {
            if(!marked[w]) {
                marked[w] = true;
                findCycle(g,w,v);
            } else if (v != u) {
                hasCycle = true;
                return;
            }
        }

    }
}
```

Java code to detect cycles in a directed graph:

```
public class DFSDirectedCycle {

    private boolean marked[];
    private boolean onStack[];
    private int s;
    private Graph g;
    private boolean hasCycle;

    public DFSDirectedCycle(Graph g, int s) {
        this.g = g;
```

```
        int.s = s;
        marked = new boolean[g.vSize()];
        onStack = new boolean[g.vSize()];
        findCycle(g,s);
    }

    public boolean hasCycle() {
        return hasCycle;
    }

    public void findCycle(Graph g, int v) {

        marked[v] = true;
        onStack[v] = true;

        for (int w : g.adjacentNodes(v)) {
            if(!marked[w]) {
                findCycle(g,w);
            } else if (onStack[w]) {
                hasCycle = true;
                return;
            }
        }

        onStack[v] = false;
    }
}
```

graph cycle depth-first-search adjacency-list dfs

edited Nov 13 '14 at 8:50

asked Oct 1 '13 at 9:55

Ivan Voroshilin
984 1 4 27

I think there is a mistake in the ordered graph code. In case of traversing a graph in the shape of O, with the root on the top and with all the edges directed to bottom, this algorithm will detect cycle (firstly will traverse the left side of O to bottom and mark all nodes as marked, then the right part of O until I will get to bottom, which is already marked). I would add there marked[v]= false; just after findCycle(g,w); What do you think? – Matej Briškár Apr 6 at 10:41

4 Answers

Answering my question:

The graph has a cycle if and only if there exists a back edge. A back edge is an edge that is from a node to itself (selfloop) or one of its ancestor in the tree produced by DFS forming a cycle.

Both approaches above actually mean the same. However, this method can be applied only to **undirected graphs**.

The reason why this algorithm doesn't work for directed graphs is that in a directed graph 2 different paths to the same vertex don't make a cycle. For example: A-->B, B-->C, A-->C - don't make a cycle whereas in undirected ones: A--B, B--C, C--A does.

Find a cycle in undirected graphs

An undirected graph has a cycle if and only if a depth-first search (DFS) finds an edge that points to an already-visited vertex (a back edge).

Find a cycle in directed graphs

In addition to visited vertices we need to keep track of vertices currently in recursion stack of function for DFS traversal. If we reach a vertex that is already in the recursion stack, then there is a cycle in the tree.

Update: Working code is in the question section above.

edited Nov 13 '14 at 8:53

answered Oct 1 '13 at 11:34

Ivan Voroshilin
984 1 4 27

In undirected graph there exists a cycle only if there is a back edge excluding the parent of the edge from where the back edge is found.Else every undirected graph has a cycle by default if we don't exclude the parent edge when finding a back edge. – whokares Jan 11 at 16:51

For the sake of completion, it is possible to find cycles in a directed graph using DFS (from wikipedia):

```
 L ← Empty list that will contain the sorted nodes
while there are unmarked nodes do
    select an unmarked node n
    visit(n)
```

```
function visit(node n)
    if n has a temporary mark then stop (not a DAG)
    if n is not marked (i.e. has not been visited yet) then
        mark n temporarily
        for each node m with an edge from n to m do
            visit(m)
        mark n permanently
        unmark n temporarily
        add n to head of L
```

edited Mar 10 at 16:13

answered Dec 18 '14 at 17:45

Igor Zelaya
2,397 2 14 39

Add wiki source! – alap Feb 20 at 12:47

@alap - Wiki source added! – Igor Zelaya Mar 10 at 16:15

Here is the code I've written in C based on DFS to find out whether a given **undirected graph** is connected/cyclic or not. with some sample output at the end. Hope it'll be helpful :)

```
#include<stdio.h>
#include<stdlib.h>

/****Global Variables****/
int A[20][20],visited[20],count=0,n;
int seq[20],connected=1,acyclic=1;

/****DFS Function Declaration****/
void DFS();

/****DFSearch Function Declaration****/
void DFSearch(int cur);

/****Main Function****/
int main()
   {
    int i,j;

    printf("\nEnter no of Vertices: ");
    scanf("%d",&n);

    printf("\nEnter the Adjacency Matrix(1/0):\n");
    for(i=1;i<=n;i++)
            for(j=1;j<=n;j++)
        scanf("%d",&A[i][j]);

    printf("\nThe Depth First Search Traversal:\n");

    DFS();

    for(i=1;i<=n;i++)
        printf("%c,%d\t",'a'+seq[i]-1,i);

    if(connected && acyclic)    printf("\n\nIt is a Connected, Acyclic Graph!");
    if(!connected && acyclic)   printf("\n\nIt is a Not-Connected, Acyclic Graph!");
    if(connected && !acyclic)   printf("\n\nGraph is a Connected, Cyclic Graph!");
    if(!connected && !acyclic)  printf("\n\nIt is a Not-Connected, Cyclic Graph!");

    printf("\n\n");
    return 0;
   }

/****DFS Function Definition****/
void DFS()
    {
    int i;
    for(i=1;i<=n;i++)
        if(!visited[i])
          {
        if(i>1) connected=0;
        DFSearch(i);
              }
    }

/****DFSearch Function Definition****/
void DFSearch(int cur)
    {
    int i,j;
    visited[cur]=++count;

        seq[count]=cur;
        for(i=1;i<count-1;i++)
                if(A[cur][seq[i]])
                   acyclic=0;

    for(i=1;i<=n;i++)
        if(A[cur][i] && !visited[i])
           DFSearch(i);

    }
```

Sample Output:

```
majid@majid-K53SC:~/Desktop$ gcc BFS.c

majid@majid-K53SC:~/Desktop$ ./a.out
***********************************

Enter no of Vertices: 10

Enter the Adjacency Matrix(1/0):

0 0 1 1 1 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 1 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0 0 0

The Depdth First Search Traversal:
a,1 c,2 d,3 f,4 b,5 e,6 g,7 h,8 i,9 j,10

It is a Not-Connected, Cyclic Graph!


majid@majid-K53SC:~/Desktop$ ./a.out
***********************************

Enter no of Vertices: 4

Enter the Adjacency Matrix(1/0):
0 0 1 1
0 0 1 0
1 1 0 0
0 0 0 1

The Depth First Search Traversal:
a,1 c,2 b,3 d,4

It is a Connected, Acyclic Graph!


majid@majid-K53SC:~/Desktop$ ./a.out
***********************************

Enter no of Vertices: 5

Enter the Adjacency Matrix(1/0):
0 0 0 1 0
0 0 0 1 0
0 0 0 0 1
1 1 0 0 0
0 0 1 0 0

The Depth First Search Traversal:
a,1 d,2 b,3 c,4 e,5

It is a Not-Connected, Acyclic Graph!

*/
```

edited May 17 at 17:22

answered May 17 at 0:42

Majid NK
23 5

Here is the code I've written in C based on DFS to find out whether a given **directed graph** is cyclic or not,i.e is there any back edge or not. with some sample output at the end. Hope it'll be helpful :)

```
#include<stdio.h>
#include<stdlib.h>

/****Global Variables****/
int A[20][20],visited[20],count=0,n;
int seq[20],s,stack[20],ptr;

/****DFS Function Declaration****/
void DFS();

/****DFSearch Function Declaration****/
void DFSearch(int cur);

/****Main Function****/
int main()
   {
    int i,j;

    printf("\nEnter no of Vertices: ");
    scanf("%d",&n);

    printf("\nEnter the Adjacency Matrix(1/0):\n");
    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
```

```
            scanf("%d",&A[i][j]);

    DFS();

    printf("\nGraph is DAG. No back edge!\n");

    printf("\n\n");
    return 0;
   }

/****DFS Function Definition****/
void DFS()
    {
      int i;
      for(i=1;i<=n;i++)
          if(!visited[i])
             DFSearch(i);
    }

/****DFSearch Function Definition****/
void DFSearch(int cur)
    {
      int i,j;

      visited[cur]=1;
      stack[++ptr]=cur;

      for(i=1;i<=n;i++)
          if(A[cur][i] && !visited[i])
             DFSearch(i);

    //when we are out of for loop it means it's a dead ends...
    while(ptr)
         {
            seq[++s]=stack[ptr--];     //...so pop the vertices from stack

            /*When a vertex v is poped off a DFS stack, no vertex u with an edge from u to
v
            can be among the vertices popped off before v. otherwise there is a backedge*/
            for(i=1;i<s;i++)
                if(A[seq[i]][seq[s]])  //if there is any back edge, exit
                  {
                    printf("\nGraph is not a DAG. There is back edge!\n\n");
                    exit(0);
                  }
          }
    }
```

Sample Output:

```
majid@majid-K53SC:~./a.out

Enter no of Vertices: 5

Enter the Adjacency Matrix(1/0):
0 0 1 0 0
0 0 1 0 1
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0

Graph is DAG. No back edge!


majid@majid-K53SC:~./a.out

Enter no of Vertices: 5

Enter the Adjacency Matrix(1/0):
0 0 1 0 0
0 0 1 0 0
0 0 0 1 1
0 0 0 0 1
0 1 0 0 0

Graph is not a DAG. There is back edge!
```

answered May 17 at 17:51

Majid NK
23 5

Please vote for me if you find it useful... – Majid NK May 17 at 17:54